04. Déclencheur différé

Imaginons la règle métier suivante dans une base de données qui gère des commandes :

> **Il doit toujours y avoir au moins un produit dans une commande.**

Considérons deux tables simples :

```
orders(id)
order_items(id, order_id)
```

Si vous tentez d'implémenter cette règle à l'aide d'un trigger standard sur la table `orders`, vous rencontrerez un problème. En effet, la création de la commande et de ses produits se fait obligatoirement en deux étapes : un premier INSERT pour créer la commande, puis un second INSERT pour créer les lignes de commande (`order_items`). Du coup, votre trigger s'exécutera après le premier INSERT, mais avant le second. Il déclenchera donc une exception, car il n'y a pas encore de produit dans la commande à ce moment-là.

Pour contourner ce problème, vous devez utiliser un **déclencheur différé**. Ce type de déclencheur est exécuté après la validation (*commit*) de la transaction, c'est-à-dire après que tous les INSERT, UPDATE ou DELETE qui en font partie ont été exécutés. L'appel à un *endpoint* étant transactionnel, cela permet de résoudre le problème.

> Note : En SQL, une transaction est un ensemble d'opérations qui sont exécutées comme une seule unité de travail. Si l'une des opérations échoue, toutes les autres sont annulées. Cela garantit que la base de données reste dans un état cohérent. Une transaction est généralement délimitée par les commandes `BEGIN` et `COMMIT`. Dans le cas d'un appel à un *endpoint*, la transaction est automatiquement gérée par PostgREST.

Voici comment créer un déclencheur différé sur la table `orders` :

```
create or replace function check_order_has_items() returns trigger
    set search_path from current as
$$
begin
    -- TODO: mettre ici le code de vérification de votre trigger
end
$$ language plpgsql security definer;

create constraint trigger check_order_has_items
    after insert
    on orders
    deferrable initially deferred
```

```
    for each row
execute procedure check_order_has_items();
```

Observations :

- Un déclencheur différé doit toujours être `after` et `for each row`.
- Le mot-clé `constraint` permet de définir un déclencheur comme une contrainte d'intégrité, ce qui autorise son exécution différée (deferrable) à la fin de la transaction.
- Les mots-clés `deferrable initially deferred` indiquent que le déclencheur doit être différé et exécuté à la fin de la transaction qui englobe la commande qui a déclenché le trigger, plutôt qu'immédiatement après l'insertion ou la mise à jour de chaque ligne provoquées par cette commande.
- La fonction du trigger doit être définie en `security definer`, car un constraint trigger différé s'exécute à la fin de la transaction (au COMMIT). À ce moment-là : la fonction *endpoint* (déclarée en `security definer`) a déjà terminé son exécution et `current_user` est revenu au rôle impersonné par PostgREST (le rôle JWT) qui n'a pas accès aux tables, ce qui déclenche du coup une exception `permission denied`. Notez que les fonctions des triggers non différés n'ont pas ce problème et peuvent être définis en `security invoker` (le défaut), car ils sont exécutés dans le contexte d'exécution du *endpoint* et sont dès lors couverts par le `security definer` du *endpoint*.

Pour plus d'informations sur les déclencheurs différés, vous pouvez consulter la documentation officielle de PostgreSQL sur les déclencheurs.